PYTHON KURSU • HAFTA 12 • GÜN 1

Pandas 4: Veri Temizleme ve Dönüştürme

Ders 23 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Bir veri setindeki sorunları (eksik, tekrar, tutarsız, aykırı) sistemli biçimde bulabileceksiniz.
- Metin ve sayı sütunlarını temizleyip doğru tiplere çevirebileceksiniz.
- Eksik değerler için silme ve doldurma stratejilerinden uygun olanı seçebileceksiniz.
- Yeni özellikler türetip pd.cut ile sınıflara ayırabileceksiniz.
- Normalleştirme ve get_dummies ile veriyi makine öğrenmesine hazırlayabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 22’in tekrarı
0:10–1:25	75 dk	Blok 1: neden temizlik • ilk bakış • tekrarlar • metin temizliği • sayıya çevirme • eksik değer stratejileri • aykırı değerler
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: tarih ve tipler • yeni özellikler • pd.cut • normalleştirme • get_dummies • temizleme fonksiyonu • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 22'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ne verir?

```
df.groupby("Mağaza")["Ciro"].sum()
```

SORU 2 • Fark ne?

```
groupby(...)["Ciro"].sum()
groupby(...)["Ciro"].transform("sum")
```

SORU 3 • Hangi how?

```
satışlar ↔ ürünler
eşleşmeyen satış satırı kalsın
```

SORU 4 • Ne ise yarar?

```
df.pivot_table(index="Mağaza",
  columns="Kategori", values="Ciro",
  aggfunc="sum")
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 2: harcama analizi

```
harcama["tarih"] = pd.to_datetime(harcama["tarih"])
kategori = harcama.groupby("kategori")["tutar"].sum()
pay = (kategori / kategori.sum() * 100).round(1)
haftalik = harcama.set_index("tarih")["tutar"].resample("W").sum()
harcama["gun_tipi"] = harcama["tarih"].dt.dayofweek.map(
    lambda g: "Hafta sonu" if g ⩾ 5 else "Hafta içi")
print(harcama.groupby("gun_tipi")["tutar"].sum())
```

SIK SORUN

Tarih metin kaldı

resample ve dt hata verdi → önce to_datetime.

SIK SORUN

“market” ve “Market”

Aynı kategori iki kez sayıldı → bugünün konusu!

BLOK 1 • 0:10–1:25 • 75 DK

Veriyi temizlemek

12 dk **Neden ve ilk bakış**	25 dk **Tekrar, metin, sayı**	28 dk **Eksik ve aykırı**	10 dk **Hatalar**

BLOK 1 • NEDEN VE ILK BAKIŞ • 12 DK

Kirli veri, yanlış sonuç

Benzetme: çürük domates

En iyi aşçı bile çürük malzemeyle iyi yemek yapamaz. İngilizcede “garbage in, garbage out” denir: modele ya da rapora ne verirseniz, ondan iyisi çıkmaz.

Sorun	Örnek (bugünkü veri)
tekrar eden satır	aynı ilan iki kez
tutarsız yazım	Kadıköy / kadıköy / Kadikoy
yanlış tip	fiyat = “4.250.000 TL” (metin)
eksik değer	metrekare, bina yaşı, fiyat boş
aykırı değer	metrekare = 9999
biçim farkı	“3+1” / “3 + 1”

BLOK 1 • NEDEN VE ILK BAKIŞ • 12 DK

İlk bakış: 135 kurgusal ilan

```
print(df.shape)
print(df.head(5).to_string())
```

```
ÇIKTI
(135, 9)
   ilan_no      ilce  oda  metrekare  bina_yasi  kat asansor          fiyat ilan_tarihi
0     1001  Beşiktaş  3+1      128.0       27.0    6     Var  11.940.000 TL  2026-09-06
1     1002   Maltepe  2+1      108.0       34.0    0     Var   5.470.000 TL  2026-10-06
2     1003   Maltepe  1+1       91.0       13.0    0     Var   5.220.000 TL  2026-09-16
3     1004   kadıköy  3+1      157.0       27.0    2     Yok  12.620.000 TL  2026-09-01
4     1005  Esenyurt  4+1     9999.0       34.0    1     Var   4.320.000 TL  2026-10-12
```

Şimdiden görülenler: **kadıköy** küçük harfle, **9999.0** metrekare, fiyat metin (“TL” ve noktalar).

BLOK 1 • NEDEN VE ILK BAKIŞ • 12 DK

Keşif: dört komutla sorun listesi

```
print(df.dtypes.astype(str).to_dict())
print("Eksikler:", df.isna().sum()[lambda s: s > 0].to_dict())
print("Tekrar eden satır:", df.duplicated().sum())
print(df["ilce"].value_counts().to_dict())
print(df["asansor"].value_counts(dropna=False).to_dict())
```

```
ÇIKTI
{'ilan_no': 'int64', 'ilce': 'str', 'oda': 'str', 'metrekare': 'float64', 'bina_yasi': 'float64', 'kat': 'int64',
  'asansor': 'str', 'fiyat': 'str', 'ilan_tarihi': 'str'}
Eksikler: {'metrekare': 7, 'bina_yasi': 5, 'asansor': 3, 'fiyat': 4}
Tekrar eden satır: 5
{'Beşiktaş': 30, 'Maltepe': 29, 'Üsküdar': 27, 'Esenyurt': 21, 'Kadıköy': 16, ' Üsküdar': 4, 'kadıköy': 3, 'Besiktas':
  3, 'Kadikoy': 1, 'Kadıköy ': 1}
{'Var': 98, 'Yok': 22, 'yok': 5, 'var': 4, 'VAR': 3, nan: 3}
```

BLOK 1 • TEKRAR, METIN, SAYI • 25 DK

Tekrar eden satırları silmek

```
print("Önce:", df.shape, "· tekrar:", df.duplicated().sum())
print("ilan_no tekrarı:", df["ilan_no"].duplicated().sum())
df = df.drop_duplicates()
print("Sonra:", df.shape)
```

```
ÇIKTI
Önce: (135, 9) · tekrar: 5
ilan_no tekrarı: 5
Sonra: (130, 9)
```

Komut	Ne yapar?
`duplicated()`	daha önce görülmüş satırlar True
`duplicated(subset=["ilan_no"])`	yalnızca bu sütuna göre
`drop_duplicates(keep="first")`	ilk görüneni tutar, sonrakileri siler

BLOK 1 • TEKRAR, METIN, SAYI • 25 DK

Metin temizliği: boşluk, harf, yazım

```
print(sorted(df["ilce"].unique().tolist()))
duzelt = {"kadikoy": "kadıköy", "besiktas": "beşiktaş"}
df["ilce"] = df["ilce"].str.strip().map(tr_kucuk).replace(duzelt).str.capitalize()
print(df["ilce"].value_counts().to_dict())
```

```
ÇIKTI
[' Üsküdar', 'Besiktas', 'Beşiktaş', 'Esenyurt', 'Kadikoy', 'Kadıköy', 'Kadıköy ', 'Maltepe', 'kadıköy', 'Üsküdar']
{'Beşiktaş': 32, 'Üsküdar': 31, 'Maltepe': 27, 'Esenyurt': 21, 'Kadıköy': 19}
```

Sıra: **strip** (boşluk) → **Türkçe küçük harf** → **düzeltme sözlüğü** → **capitalize** (tek biçim).

BLOK 1 • TEKRAR, METIN, SAYI • 25 DK

Biçim birliği: “3 + 1” → “3+1”, “VAR” → True

```
print(df["oda"].unique().tolist())
df["oda"] = df["oda"].str.replace(" ", "")
df["asansor"] = df["asansor"].str.strip().str.lower().map({"var": True, "yok": False})
print(df["oda"].value_counts().to_dict())
print(df["asansor"].value_counts(dropna=False).to_dict())
```

```
ÇIKTI
['3+1', '2+1', '1+1', '4+1', '3 + 1', '4 + 1', '2 + 1']
{'2+1': 61, '3+1': 40, '4+1': 15, '1+1': 14}
{True: 102, False: 25, nan: 3}
```

BLOK 1 • TEKRAR, METIN, SAYI • 25 DK

Metinden sayıya: “4.250.000 TL” → 4250000

```
print(df["fiyat"].head(4).tolist())
temiz = df["fiyat"].str.replace(" TL", "").str.replace(".", "")
df["fiyat"] = pd.to_numeric(temiz, errors="coerce")
print(df["fiyat"].dtype, "· eksik:", df["fiyat"].isna().sum())
print(df["fiyat"].describe().round(0).to_dict())
```

```
ÇIKTI
['11.940.000 TL', '5.470.000 TL', '5.220.000 TL', '12.620.000 TL']
float64 · eksik: 4
{'count': 126.0, 'mean': 7559365.0, 'std': 3291292.0, 'min': 1600000.0, '25%': 5267500.0, '50%': 7090000.0, '75%':
  9432500.0, 'max': 16610000.0}
```

to_numeric(errors="coerce"): çevrilemeyen değer hata vermez, NaN olur; sonra kaç tane olduğunu sayarız.

BLOK 1 • EKSIK VE AYKIRI • 28 DK

Eksik değerle ne yapmalı?

Strateji	Kod	Ne zaman?	Risk
satırı sil	`dropna(subset=["fiyat"])`	hedef değişken eksik; az sayıda	veri kaybı
sabit değer	`fillna(0) / fillna("bilinmiyor")`	eksik = “yok” anlamına geliyorsa	yanlış anlam
ortalama / medyan	`fillna(df["s"].median())`	sayısal, az eksik	dağılımı daraltır
grup medyanı	`groupby(…)["s"].transform("median")`	grup farkları büyükse	daha karmaşık
en sık değer	`fillna(df["s"].mode()[0])`	kategorik sütun	çoğunluğu şişirir

Altın kural: önce **neden eksik** diye sorun; kararı ve sayısını not edin.

BLOK 1 • EKSIK VE AYKIRI • 28 DK

Eksik değerleri işlemek: sil, medyan, grup medyanı

```
print("Önce:", df.isna().sum()[lambda s: s > 0].to_dict())
df = df.dropna(subset=["fiyat"])
oda_medyan = df.groupby("oda")["metrekare"].transform("median")
df["metrekare"] = df["metrekare"].fillna(oda_medyan)
df["bina_yasi"] = df["bina_yasi"].fillna(df["bina_yasi"].median())
df["asansor"] = df["asansor"].fillna(df["asansor"].mode()[0])
print("Sonra:", df.isna().sum().sum(), "eksik ·", df.shape)
```

```
ÇIKTI
Önce: {'metrekare': 6, 'bina_yasi': 5, 'asansor': 3, 'fiyat': 4}
Sonra: 0 eksik · (126, 9)
```

Aykırı değerler: IQR ile bulmak

```
print(df["metrekare"].describe()[["min", "50%", "max"]].to_dict())
q1, q3 = df["metrekare"].quantile([0.25, 0.75])
iqr = q3 - q1
alt, ust = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"IQR sınırları: {alt:.0f} – {ust:.0f}")
aykiri = (df["metrekare"] < alt) | (df["metrekare"] > ust)
print(df.loc[aykiri, ["ilan_no", "oda", "metrekare"]].to_string(index=False))
df.loc[aykiri, "metrekare"] = pd.NA
```

```
ÇIKTI
{'min': 62.0, '50%': 110.0, 'max': 9999.0}
IQR sınırları: 35 – 188
 ilan_no oda  metrekare
    1005 4+1     9999.0
    1018 3+1     9999.0
```

BLOK 1 • SIK YAPILAN HATALAR

Temizlikte sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`df["fiyat"].astype(float) # "TL" var`	`ValueError: could not convert string to float: '4.250.000 TL'`	önce temizle / to_numeric
`df["oda_sayisi"].astype(int) # NaN var`	`pandas.errors.IntCastingNaNError: Cannot convert non-finite values (NA or inf) to integer. Replace or remove non-finite values or cast to an integer type that supports these values (e.g. 'Int64')`	önce fillna ya da "Int64"
`sayısal sütunda .str`	`AttributeError: Can only use .str accessor with string values, not floating. Did you mean: 'std'?`	yalnızca metin sütunlarında

Hata vermeyen tuzak

dropna() — subset'siz: çok satır gider
ortalama ile doldurmak (aykırılar varken)
df["s"].fillna(…) sonucunu atamamak

Doğrusu

`dropna(subset=[…])`
`önce aykırılar, sonra medyan`
`df["s"] = df["s"].fillna(…)`

1:25–1:40 • 15 DK

Mola

Moladan sonra: temiz veriden yeni bilgiler türetmek ve makinenin anlayacağı sayılara çevirmek

BLOK 2 • 1:40–2:55 • 75 DK

Veriyi dönüştürmek

20 dk
Tipler ve yeni özellikler

25 dk
Ölçekleme ve kodlama

15 dk
Temizleme fonksiyonu

15 dk
Alıştırmalar

BLOK 2 • TIPLER VE YENI ÖZELLIKLER • 20 DK

Doğru tipler: tarih, kategori, tamsayı

```
df["ilan_tarihi"] = pd.to_datetime(df["ilan_tarihi"])
df["ilce"] = df["ilce"].astype("category")
df["bina_yasi"] = df["bina_yasi"].astype(int)
print(df.dtypes.astype(str).to_dict())
print(df["ilce"].cat.categories.tolist())
```

```
ÇIKTI
{'ilan_no': 'int64', 'ilce': 'category', 'oda': 'str', 'metrekare': 'float64', 'bina_yasi': 'int64', 'kat': 'int64',
  'asansor': 'bool', 'fiyat': 'float64', 'ilan_tarihi': 'datetime64[us]'}
['Beşiktaş', 'Esenyurt', 'Kadıköy', 'Maltepe', 'Üsküdar']
```

BLOK 2 • TIPLER VE YENI ÖZELLIKLER • 20 DK

Yeni özellikler: oda sayısı, m² fiyatı, ilan yaşı

```
df["oda_sayisi"] = df["oda"].str.split("+").str[0].astype(int)
df["m2_fiyat"] = (df["fiyat"] / df["metrekare"]).round(0)
df["ilan_gunu"] = (pd.Timestamp("2026-10-31") - pd.to_datetime(df["ilan_tarihi"])).dt.days
df["bodrum"] = df["kat"] < 0
print(df[["oda", "oda_sayisi", "m2_fiyat", "ilan_gunu", "bodrum"]].head(4).to_string(index=False))
print(df.groupby("ilce")["m2_fiyat"].median().sort_values(ascending=False).to_dict())
```

```
ÇIKTI
oda  oda_sayisi  m2_fiyat  ilan_gunu  bodrum
3+1           3   93281.0         55   False
2+1           2   50648.0         25   False
1+1           1   57363.0         45   False
3+1           3   80382.0         60   False
{'Beşiktaş': 96269.0, 'Kadıköy': 86627.0, 'Üsküdar': 68819.0, 'Maltepe': 56356.0, 'Esenyurt': 28143.0}
```

BLOK 2 • TIPLER VE YENI ÖZELLIKLER • 20 DK

pd.cut ve pd.qcut: sayıyı sınıflara ayırmak

```
df["yas_grubu"] = pd.cut(df["bina_yasi"], bins=[-1, 5, 15, 30, 100],
                         labels=["Yeni (0-5)", "Genç (6-15)", "Orta (16-30)", "Eski (31+)"])
print(df["yas_grubu"].value_counts().sort_index().to_dict())
df["fiyat_dilimi"] = pd.qcut(df["fiyat"], q=3, labels=["ucuz", "orta", "pahalı"])
print(df.groupby("fiyat_dilimi", observed=True)["fiyat"].agg(["min", "max"]).round(-4))
```

```
ÇIKTI
{'Yeni (0-5)': 12, 'Genç (6-15)': 24, 'Orta (16-30)': 56, 'Eski (31+)': 34}
                    min         max
fiyat_dilimi
ucuz          1600000.0   5920000.0
orta          6000000.0   8660000.0
pahalı        8740000.0  16610000.0
```

BLOK 2 • ÖLÇEKLEME VE KODLAMA • 25 DK

Normalleştirme neden gerekli?

Benzetme: farklı birimler

Bir yarışmada boyu santimetre, kiloyu ton cinsinden yazarsanız boy farkları her şeyi ezer. Farklı ölçekteki sütunları ortak bir ölçeğe getirmek, hiçbirinin “sesini” fazla yükseltmemek demektir.

Sütun	Tipik aralık
fiyat	`≈ 1.000.000 – 20.000.000`
metrekare	`≈ 40 – 220`
bina_yasi	`0 – 39`
oda_sayisi	`1 – 4`

Yöntem	Formül	Sonuç aralığı
Min-max	`(x - min) / (max - min)`	0 – 1
Z-skor (standartlaştırma)	`(x - ortalama) / std`	ortalama 0, std 1

BLOK 2 • ÖLÇEKLEME VE KODLAMA • 25 DK

Min-max ve z-skor: formülle, bütün sütunlara birden

```
df["oda_sayisi"] = df["oda"].str.split("+").str[0].astype(int)
sutunlar = ["metrekare", "bina_yasi", "oda_sayisi"]
enk, enb = df[sutunlar].min(), df[sutunlar].max()
minmax = (df[sutunlar] - enk) / (enb - enk)
zskor = (df[sutunlar] - df[sutunlar].mean()) / df[sutunlar].std()
print(minmax.describe().loc[["min", "max"]].round(2))
print(zskor.describe().loc[["mean", "std"]].round(2))
```

```
ÇIKTI
metrekare  bina_yasi  oda_sayisi
min         0.0         0.0
  0.0
max         1.0         1.0
  1.0
metrekare  bina_yasi  oda_sayisi
mean         0.0         0.0
  0.0
std          1.0         1.0
  1.0
```

BLOK 2 • ÖLÇEKLEME VE KODLAMA • 25 DK

get_dummies: kategorik sütunu sayılara çevirmek

```
kodlu = pd.get_dummies(df[["ilce", "asansor", "metrekare"]], columns=["ilce"], prefix="ilce", dtype=int)
print(kodlu.head(3).to_string())
kodlu2 = pd.get_dummies(df["ilce"], prefix="ilce", drop_first=True, dtype=int)
print(kodlu2.columns.tolist())
```

```
ÇIKTI
   asansor  metrekare  ilce_Beşiktaş  ilce_Esenyurt  ilce_Kadıköy  ilce_Maltepe  ilce_Üsküdar
0     True      128.0              1              0             0             0             0
1     True      108.0              0              0             0             1             0
2     True       91.0              0              0             0             1             0
['ilce_Esenyurt', 'ilce_Kadıköy', 'ilce_Maltepe', 'ilce_Üsküdar']
```

Makine öğrenmesi modelleri metin anlamaz: “Kadıköy” yerine **ilce_Kadıköy = 1**. Ayrıntısı Ders 29–30’da.

BLOK 2 • ÖLÇEKLEME VE KODLAMA • 25 DK

Sıralı kategoriler: map ile sayıya çevirmek

```
sira = {"Yeni (0-5)": 0, "Genç (6-15)": 1,
        "Orta (16-30)": 2, "Eski (31+)": 3}
sinirlar = [-1, 5, 15, 30, 100]
df["yas_grubu"] = pd.cut(df["bina_yasi"], sinirlar, labels=list(sira))
df["yas_kodu"] = df["yas_grubu"].map(sira).astype(int)
ornek = df.drop_duplicates("yas_grubu").sort_values("yas_kodu")
print(ornek[["bina_yasi", "yas_grubu", "yas_kodu"]].to_string(index=False))
```

```
ÇIKTI
 bina_yasi    yas_grubu  yas_kodu
       5.0   Yeni (0-5)         0
      13.0  Genç (6-15)         1
      27.0 Orta (16-30)         2
      34.0   Eski (31+)         3
```

Sıralı (yaş grubu, eğitim düzeyi, beden S-M-L) → **map** ile 0, 1, 2… • **Sırasız** (ilçe, renk, marka) → **get_dummies**

BLOK 2 • TEMIZLEME FONKSIYONU • 15 DK

Uygulama: bütün adımlar tek fonksiyonda

```
def temizle(df):
    df = df.drop_duplicates().copy()
    duzelt = {"kadikoy": "kadıköy", "besiktas": "beşiktaş"}
    df["ilce"] = df["ilce"].str.strip().map(tr_kucuk).replace(duzelt).str.capitalize()
    df["oda"] = df["oda"].str.replace(" ", "")
    df["asansor"] = df["asansor"].str.strip().str.lower().map({"var": True, "yok": False})
    df["fiyat"] = pd.to_numeric(df["fiyat"].str.replace(" TL", "").str.replace(".", ""),
                                errors="coerce")
    df = df.dropna(subset=["fiyat"])
    df.loc[df["metrekare"] > 1000, "metrekare"] = pd.NA            # aykırı → eksik
    df["metrekare"] = df["metrekare"].fillna(df.groupby("oda")["metrekare"].transform("median"))
    df["bina_yasi"] = df["bina_yasi"].fillna(df["bina_yasi"].median()).astype(int)
    df["asansor"] = df["asansor"].fillna(df["asansor"].mode()[0]).astype(bool)
    df["ilan_tarihi"] = pd.to_datetime(df["ilan_tarihi"])
    df["oda_sayisi"] = df["oda"].str.split("+").str[0].astype(int)
    df["m2_fiyat"] = (df["fiyat"] / df["metrekare"]).round(0)
    return df.reset_index(drop=True)
```

BLOK 2 • TEMIZLEME FONKSIYONU • 15 DK

Önce ve sonra

```
ÇIKTI
Ham: (135, 9) · eksik: 19 · tekrar: 5
Temiz: (126, 11) · eksik: 0 · tekrar: 0
{'Üsküdar': 31, 'Beşiktaş': 28, 'Maltepe': 27, 'Esenyurt': 21, 'Kadıköy': 19}
     metrekare       fiyat  m2_fiyat
min       62.0   1600000.0   21961.0
max      179.0  16610000.0  126087.0
ev_ilanlari_temiz.csv kaydedildi (Ders 29'da kullanılacak).
```

ÖNCE

135 satır

tekrar, 5 farklı yazımda ilçe, metin fiyat, eksik ve aykırı değerler

SONRA

temiz tablo

tekrar yok, eksik yok, doğru tipler, yeni özellikler

BLOK 2 • SIK YAPILAN HATALAR

Dönüştürmede sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`pd.cut(…, bins=[0, 5, 15], labels=["a", "b", "c"])`	`ValueError: Bin labels must be one fewer than the number of bin edges`	etiket = aralık sayısı
`oda.str.split("+").str[0].astype(int) # "x"`	`ValueError: invalid literal for int() with base 10: 'x'`	önce temizleyin

Hata vermeyen tuzak	Doğrusu
ilçeleri 1, 2, 3… diye kodlamak	sırasız → get_dummies
ölçeklemeyi bütün veriyle hesaplamak (ML)	yalnızca eğitim verisiyle (Ders 29)
pd.cut'ta sınırın dışında kalan değer	NaN olur; alt sınırı –1 gibi seçin

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 15 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Kategori temizliği

[“Market”, “market ”, “MARKET”, “Ulaşım”, “ulasim”, None] serisini temizleyin: tek biçim, eksik → “Diğer”; value_counts.

ORTA • 5 dk

2 • Fiyat metinleri

[“1.250,50 TL”, “980 TL”, “2.100,00”, “yok”] → sayı (float).
Çevrilemeyenleri sayın ve ortalamayı bulun.

ZOR • 6 dk

3 • Ev özellikleri

Temiz ilanlarda: “kat_grubu” (bodrum/giriş/ara/yüksek) pd.cut ile; ilçe başına medyan fiyat; ilçe dummies + z-skorlu metrekare tablosu.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: kategori temizliği

```
import pandas as pd


def tr_kucuk(metin):
    return metin.replace("İ", "i").replace("I", "ı").lower()


kat = pd.Series(["Market", "market ", "MARKET", "Ulaşım", "ulasim", None])
temiz = kat.str.strip().map(tr_kucuk, na_action="ignore").replace({"ulasim": "ulaşım"})
temiz = temiz.str.capitalize().fillna("Diğer")
print(temiz.value_counts().to_dict())
```

```
ÇIKTI
{'Market': 3, 'Ulaşım': 2, 'Diğer': 1}
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 2: Türkçe ondalıklı fiyatlar

```
import pandas as pd

fiyat = pd.Series(["1.250,50 TL", "980 TL", "2.100,00", "yok"])
temiz = (fiyat.str.replace(" TL", "", regex=False)
              .str.replace(".", "", regex=False)
              .str.replace(",", ".", regex=False))
sayi = pd.to_numeric(temiz, errors="coerce")
print(sayi.tolist())
print("Çevrilemeyen:", sayi.isna().sum(), "· ortalama:", round(sayi.mean(), 2))
```

```
ÇIKTI
[1250.5, 980.0, 2100.0, nan]
Çevrilemeyen: 1 · ortalama: 1443.5
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3: ev özellikleri

```
df = temizle(pd.read_csv("ev_ilanlari_kirli.csv"))
df["kat_grubu"] = pd.cut(df["kat"], bins=[-2, -1, 0, 5, 100], labels=["bodrum", "giriş", "ara", "yüksek"])
print(df["kat_grubu"].value_counts().sort_index().to_dict())
print(df.groupby("ilce")["fiyat"].median().sort_values(ascending=False).round(-4).to_dict())
model_tablosu = pd.get_dummies(df[["ilce"]], prefix="ilce", drop_first=True, dtype=int)
model_tablosu["metrekare_z"] = ((df["metrekare"] - df["metrekare"].mean()) / df["metrekare"].std()).round(2)
print(model_tablosu.head(3).to_string())
```

```
ÇIKTI
{'bodrum': 8, 'giriş': 15, 'ara': 53, 'yüksek': 50}
{'Beşiktaş': 11380000.0, 'Kadıköy': 8850000.0, 'Üsküdar': 7120000.0, 'Maltepe': 5920000.0, 'Esenyurt': 3360000.0}
   ilce_Esenyurt  ilce_Kadıköy  ilce_Maltepe  ilce_Üsküdar  metrekare_z
0              0             0             0             0         0.60
1              0             0             1             0        -0.17
2              0             0             1             0        -0.83
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- Önce keşif: **dtypes, isna().sum(), duplicated().sum(), value_counts()** → sorun listesi.
- Temizlik: **drop_duplicates**, **str.strip/lower + düzeltme sözlüğü**, **to_numeric(errors="coerce")**.
- Eksik: **dropna(subset)**, **fillna(medyan)**, grup medyanı (**transform**); aykırı: **IQR**.
- Dönüşüm: tipler, yeni özellikler, **pd.cut/qcut**, **min-max**, **z-skor**, **get_dummies**, sıralı **map**.
- Adımları bir **fonksiyonda** toplayın ve kararları not edin.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kendi veri setinizi temizleyin

Ders 21–22'deki açık veri setinize bir temizle() fonksiyonu yazın: en az 5 temizlik adımı ve her adım için bir cümlelik gerekçe. Önce/sonra özetini raporlayın.

2 • ZORUNLU

Harcama tablosu son hali

Harcama kategorilerinizi tek biçime getirin, tutarı sayıya çevirin, "tutar_dilimi" (pd.cut) ve kategori dummies ekleyin.

3 • BONUS

IQR fonksiyonu

aykiri_bul(seri) fonksiyonu yazın: IQR sınırlarını ve aykırı değerlerin index'lerini döndürsün; ilan verisinde fiyat ve m2_fiyat için çalıştırın.

Teslim: ders23_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 12 • GÜN 2

Matplotlib: veriyi görselleştirmek

Bugün temizlediğimiz tabloyu gelecek ders grafiğe dökeceğiz:

- plot, scatter, bar, pie, histogram
- Başlık, eksen, renk, açıklama (legend)
- Alt grafikler (subplots)
- Pandas ile doğrudan çizim ve kaydetme

Önizleme

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("ev_ilanlari_temiz.csv")
df.groupby("ilce")["m2_fiyat"].median().sort_values().plot(kind="barh")
plt.title("İlçelere göre medyan m² fiyatı (kurgusal veri)")
plt.tight_layout()
plt.savefig("onizleme.png")
```

```
(onizleme.png kaydedildi)
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
Eksik veriyle çalışmak	pandas.pydata.org/docs/user_guide/missing_data.html
Metin sütunları (.str)	pandas.pydata.org/docs/user_guide/text.html
pd.cut ve kategorik veri	pandas.pydata.org/docs/user_guide/categorical.html
get_dummies	pandas.pydata.org/…/pandas.get_dummies.html